

作業ログ

報告者：佐藤涼菜

報告日：6 月 23 日

プロジェクト名：マジカル☆観覧車

1. 進捗サマリー（要約）

- 全体進捗率：85[%]
- マイルストーン達成状況：おおよそ達成（テストフェーズは結合待ち）
- 主要成果：
 - 指揮デバイスの最終固定および加工をし，本体・センサの実装（No.2112）を完了
 - BPM・音量変化機構の実装（No.2113）を完了
 - 計画書との変更点の記述（No.2231）
- 重大課題（影響度：高）：
 - テストフェーズ（No.3241）が結合待ちのため未着手

2. 作業ログ（詳細トラッキング）

2.1 作業ログ

表 1: 作業ログ

No.	作業項目	開始日	予定完了日	状態	進捗率	依存関係・ブロッカー	コメント
2112	本体・センサの実装	5/24	6/17	完了	100%	無	回路の最終固定と筐体の加工を行った。
2113	BPM・音量変化機構の実装	5/29	6/1	完了	100%	無	スライド式可変抵抗器の動作確認を実施し完了。
3241	LED マトリックスの反映確認	6/17	6/30	未着手	0%	有	結合テストの実施待ち。6/24 の授業内にて実施予定。
-	作業ログの記入	6/17	6/23	完了	100%	無	特になし。

2.2 日ごとの作業ログ

表 2: 日ごとの作業ログ

日付	No.	作業項目	時間	開始時の状態	終了時の状態	進捗率	コメント
6/17	2113	BPM・音量変化機構の実装	1.5h	おおよそ完了	完了	100%	スライド式可変抵抗器の再はんだ付けおよび動作確認を実施し完了.
6/18	2112	本体・センサの実装	1h	進行中	進行中	93%	内部配線の整理と筐体を加工するための備品調達.
6/19	2112	本体・センサの実装	2h	進行中	進行中	96%	操作部の露出加工, 筐体の加工.
6/20	2112	本体・センサの実装	1.5h	進行中	進行中	98%	回路の最終固定と筐体内部の配線固定を実施.
6/21	2112	本体・センサの実装	0.5h	進行中	完了	100%	筐体の最終仕上げを行い, 本体・センサの実装を完了.
6/22	2231	LED マトリクス制御	1h	完了	完了	100%	計画書と実装の変更点を整理し記述.
6/23	-	作業ログの記入	1h	未着手	完了	100%	特になし.

3. 計画時の GC と現在の GC

計画段階のガントチャート (GC) を図 1 (実装フェーズ) および図 2 (テストフェーズ) に示す.

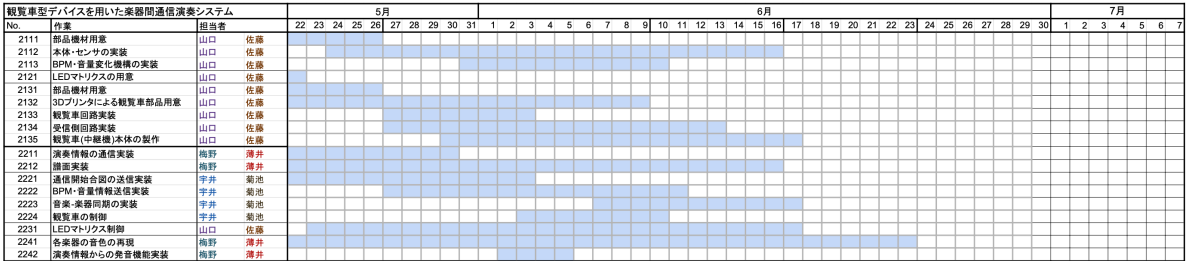


図 1 計画段階のガントチャート (実装フェーズ)

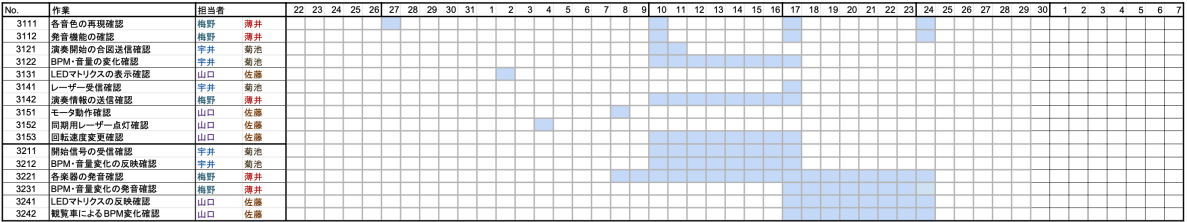


図 2 計画段階のガントチャート (テストフェーズ)

次に、現在の GC を図 3 に示す。達成した項目は緑色で塗られている。

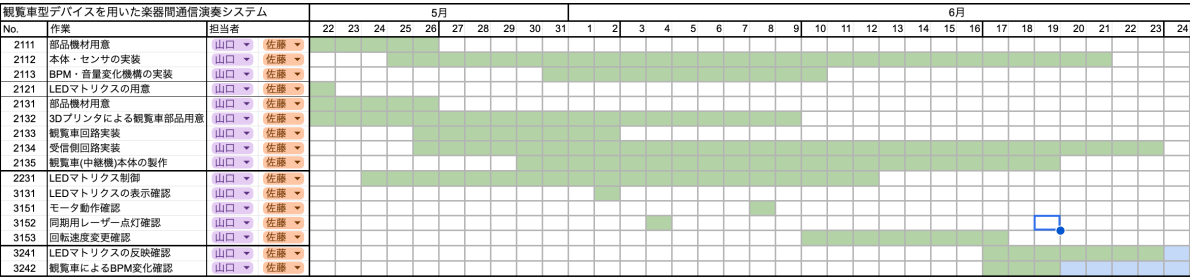


図 3 現在のガントチャート

4. 課題管理

課題	発生原因	影響度	優先度	対策	期限	状態
可変抵抗器がブレッドボード上で安定接続できない(No.2112/2113に影響)	スライド式可変抵抗器の端子形状がブレッドボードに適合しないため.	高	高	スライド式可変抵抗器を基盤に設置し、筐体へ固定した.	6/12	対応済
1 機のスライド式可変抵抗器のはんだ付け設置不良 (No.2112 に影響)	基盤へのはんだ付け時に接触不良が発生したため.	中	高	6/17 に再はんだ付けを実施し、動作を確認した.	6/17	対応済
テストフェーズが結合待ち (No.3241 に影響)	通信系との結合が完了していないため、LED マトリクスの反映確認テストが実施できない.	高	中	通信系の準備完了を待ち、結合テストを実施する.	6/30	対応中

※影響度＝プロジェクトへのインパクト

※優先度＝対応の緊急性

5. AI 利用状況と検証（再現性重視）

今回は AI を利用していない.

6. No.2231 LED マトリクス制御：計画書との変更点

6.1 描画 API の変更

計画書 (3.5.8.3) では、フレームバッファの描画に `matrix.renderBitmap()` を使用すると記述されている。`renderBitmap()` は `uint8_t[8][12]` 形式の配列を引数に取る関数である。実装では `uint32_t[3]` 形式にビットパックし、`matrix.loadFrame()` で描画する方式に変更した。これは、`loadFrame()` の方がフレームバッファのメモリ効率が高く、96 ビット (8 行 × 12 列) を 3 つの 32 ビット整数で表現できるためである。

6.2 関数の追加

計画書の表 3.33 には `checkUDP` と `displayBPM` の 2 関数のみ記載されている。実装では以下の 3 関数を追加した。

- `clearDisplay()`: 演奏終了ヘッダ ('E') 受信時に LED マトリクスを全消灯する関数。計画書の通信プロトコル (表 3.29, 3.30) にはヘッダ 'E' が定義されているが、Display 側の関数一覧にはこの処理への言及がなかったため追加した。
- `drawDigit()`: 1 桁の数字を独自フォント (`font3x5[]`) を参照してフレームバッファに描画する内部関数。`displayBPM()` から桁ごとに呼び出される。
- `displaySetup()`: `matrix.begin()` を Display.cpp 側で呼ぶための初期化関数。`Arduino_LED_Matrix` の表示バッファはヘッダ内で static 定義されており、include する翻訳単位ごとに別実体となる。そのため、描画を行う `loadFrame()` と同一ファイルから `begin()` を呼び出す必要があり、本関数を追加した。

6.3 モジュール構成の変更

計画書の図 3.16 では `checkUDP()` は Display.ino 側に配置されている。実装では Display.cpp に移動した。`checkUDP()` 内で `currentBPM` (Display.cpp の静的変数) を参照する必要があるためである。これにより、Display.ino には `setup()` と `loop()` のみが残る構成となった。

6.4 受信データの読み込み形式

計画書 (3.5.8.3) では「ペイロードを文字列として読み込む」と記述されている。実装では `uint8_t buffer[2]` としてバイナリ形式で読み取り、`buffer[0]` をヘッダ文字 (char)、`buffer[1]` を段階番号 (`uint8_t`) として処理している。通信プロトコル仕様 (表 3.29, 3.30) 自体は固定長 2 バイトのバイナリ形式で定義されており、実装側がプロトコル仕様に忠実な形式を採用した。

6.5 静的 IP 設定の追加

計画書の表 3.24 には Display 側の定数として `localIP` (観覧車デバイス自身の静的 IP アドレス) が定義されている。実装では `WiFi.config(localIP, gateway, subnet)` を `WiFi.begin()` の前に呼び出すことで、指揮デバイスの送信先 IP アドレス (192.168.11.10) と一致する固定 IP を設定した。

7. 次回計画

- 目標：通信系との結合テストの実施
 - 通信系の準備が完了次第、LED マトリクスの反映確認テスト (No.3241) を実施する。
 - 指揮デバイスからの UDP 送信により BPM 値が LED マトリクスに正しく表示されることを確認する。
- 達成基準：
 - 指揮デバイスから BPM 値を送信し、表示デバイスの LED マトリクスに正しく反映されること。
 - 演奏終了ヘッダ ('E') の受信時に LED マトリクスが消灯すること。
- 期限：
 - 2026/6/30

8. 付録

No.2231 (LED マトリクス制御) のプログラムを以下に示す。

8.1 Display.h

```
1  #ifndef DISPLAY_H
2  #define DISPLAY_H
3
4  #include <Arduino.h>
5  #include "Arduino_LED_Matrix.h" // Arduino UNO R4 WiFi 内蔵マトリクス用ライブラリLED
6  #include <WiFiS3.h> //通信UDP
7
8  // 定数定義BPM
9  #define BPM_STEP_1 60
10 #define BPM_STEP_2 90
11 #define BPM_STEP_3 120
12 #define BPM_STEP_4 150
13 #define BPM_STEP_5 180
14
15 // 通信定数
16 #define LOCAL_PORT 2390 // 受信に利用するポート番号UDP
17 #define HEADER_BPM 'B' // データ受信の識別ヘッダBPM
18 #define HEADER_END 'E' // 演奏終了の識別ヘッダ (マトリクス消灯) LED
19
20 // 文字フォントの寸法
21 #define FONT_WIDTH 3 // 文字の横ドット数1
22 #define FONT_HEIGHT 5 // 文字の縦ドット数1
23
24 // マトリクスの寸法 (LEDArduino UNO R4 WiFi 内蔵: 行×列) 812
25 #define MATRIX_ROWS 8
26 #define MATRIX_COLS 12
27
28 // 関数宣言
29
30 // マトリクスの初期化 (LEDmatrix.begin()を本関数経由で呼ぶ。
31 void displaySetup();
32
33 // 中継機からのパケット確認, 値の受信処理UDPBPM
34 void checkUDP();
35
36 // 受信した値をマトリクスに数値表示する (引数: BPMLEDint ) bpm
37 void displayBPM(int bpm);
38
39 // マトリクスを全消灯する (演奏終了時に使用) LED
40 void clearDisplay();
41
42 // 指定された桁の数字を, 共有フレームバッファの1 x_offset 列から描画する
43 void drawDigit(int digit, int x_offset);
44
45 // 外部参照
46
47 // × (横×縦) の独自数字フォント (～) 353509
48 extern const uint8_t font3x5[10][15];
49
50 // マトリクスインスタンス (LEDDisplay.cpp で定義, .ino の setup() で begin() を呼ぶ)
51 extern ArduinoLEDMatrix matrix;
52
53 // 通信オブジェクトUDP
54 extern WiFiUDP Udp;
55
56 #endif
```

8.2 Display.ino

```
1  #include "Display.h"
2
3  WiFiUDP Udp;
4
5  const char SSID[] = "hackathon001-WPA2";
6  const char PASS[] = "hackathon001";
7
8  // 静的設定 (指揮デバイスの送信先IP ferrisWheelIP2 と一致)
9  IPAddress localIP(192, 168, 11, 10);
10 IPAddress gateway(192, 168, 11, 1);
11 IPAddress subnet(255, 255, 255, 0);
12
13 void setup() {
14     displaySetup();
15
16     if (SSID[0] != '\0') {
17         WiFi.config(localIP, gateway, subnet);
18         while (WiFi.begin(SSID, PASS) != WL_CONNECTED) {
19             delay(1000);
20         }
21     }
22     Udp.begin(LOCAL_PORT);
```

```

23 }
24
25 void loop() {
26     checkUDP();
27 }

```

8.3 Display.cpp

```

1  #include "Display.h"
2  #include "Arduino_LED_Matrix.h"
3
4  // 独自フォント定義
5  // font3x5[digit][row*FONT_WIDTH + col] : 点灯1=, 消灯0=
6
7  const uint8_t font3x5[10][15] = {
8      // 0
9      {1, 1, 1,
10       1, 0, 1,
11       1, 0, 1,
12       1, 0, 1,
13       1, 1, 1},
14      // 1
15      {0, 1, 0,
16       1, 1, 0,
17       0, 1, 0,
18       0, 1, 0,
19       1, 1, 1},
20      // 2
21      {1, 1, 1,
22       0, 0, 1,
23       1, 1, 1,
24       1, 0, 0,
25       1, 1, 1},
26      // 3
27      {1, 1, 1,
28       0, 0, 1,
29       0, 1, 1,
30       0, 0, 1,
31       1, 1, 1},
32      // 4
33      {1, 0, 1,
34       1, 0, 1,
35       1, 1, 1,
36       0, 0, 1,
37       0, 0, 1},
38      // 5
39      {1, 1, 1,
40       1, 0, 0,
41       1, 1, 1,
42       0, 0, 1,
43       1, 1, 1},
44      // 6
45      {1, 1, 1,
46       1, 0, 0,
47       1, 1, 1,
48       1, 0, 1,
49       1, 1, 1},
50      // 7
51      {1, 1, 1,
52       0, 0, 1,
53       0, 0, 1,
54       0, 0, 1,
55       0, 0, 1},
56      // 8
57      {1, 1, 1,
58       1, 0, 1,
59       1, 1, 1,
60       1, 0, 1,
61       1, 1, 1},
62      // 9
63      {1, 1, 1,
64       1, 0, 1,
65       1, 1, 1,
66       0, 0, 1,
67       1, 1, 1}
68 };
69
70 // bpmTable[stage-1] に対応するを得るBPM
71 static const int bpmTable[5] = {
72     BPM_STEP_1, // 60
73     BPM_STEP_2, // 90
74     BPM_STEP_3, // 120
75     BPM_STEP_4, // 150

```

```

76   BPM_STEP_5    // 180
77 };
78
79 // マトリクスのインスタンス (行×列) LED812
80 ArduinoLEDMatrix matrix;
81
82 // (マトリクスの初期化) displaySetupLED
83 // Arduino_LED_Matrix の表示バッファ(framebuffer)はヘッダ内で static 定義
84 // されており, include する翻訳単位(.ino / .cpp)ごとに別実体となる. そのため
85 // 表示を駆動する割り込みを登録する begin() と, 描画を行う loadFrame() は
86 // 必ず同一ファイル本( Display.cpp)から呼び出し, 同じ framebuffer を共有させる.
87 // .ino からは matrix を直接触らず, 本関数と displayBPM() を呼ぶ
88
89 void displaySetup() {
90     matrix.begin();
91
92     // begin() 直後の最初の loadFrame は反映されない場合があるため,
93     // 消灯フレームを一度描画ウォームアップ()してから安定するまで待機する.
94     uint32_t warmUp[3] = {0, 0, 0};
95     matrix.loadFrame(warmUp);
96     delay(200);
97 }
98
99 // 描画用共有フレームバッファ (行MATRIX_ROWS ×列, MATRIX_COLS1点灯= / 消灯) 0=
100 // drawDigit() がここに書き込み, displayBPM() がまとめてレンダリングする
101 static uint8_t frameBuffer[MATRIX_ROWS][MATRIX_COLS];
102
103 // 直近に表示中の値 (更新検知用: 値が変化した時だけ再描画する) BPM
104 static int currentBPM = -1;
105
106 // checkUDP
107 // bufferはヘッダ[0]('Bとか')
108 // bufferは段階番号[1]
109
110 void checkUDP() {
111     int packetSize = Udp.parsePacket();
112     if (packetSize <= 0) return;
113
114     uint8_t buffer[2] = {0};
115     int len = Udp.read(buffer, sizeof(buffer));
116     if (len < 2) return;
117
118     char header = (char)buffer[0];
119     uint8_t stage = buffer[1];
120
121     if (header == HEADER_END) {
122         clearDisplay();
123         return;
124     }
125
126     if (header != HEADER_BPM) return;
127     if (stage < 1 || stage > 5) return;
128
129     int bpm = bpmTable[stage - 1];
130
131     if (bpm != currentBPM) {
132         currentBPM = bpm;
133         displayBPM(bpm);
134     }
135 }
136
137 // displayBPM
138
139 void displayBPM(int bpm) {
140     // フレームバッファをすべて消灯にリセット
141     memset(frameBuffer, 0, sizeof(frameBuffer));
142
143     // 値を桁配列に分解BPM
144     int digits[3]; // 最大桁分3
145     int numDigits;
146
147     if (bpm >= 100) {
148         numDigits = 3;
149         digits[0] = bpm / 100; // 百の位
150         digits[1] = (bpm / 10) % 10; // 十の位
151         digits[2] = bpm % 10; // 一の位
152     } else if (bpm >= 10) {
153         numDigits = 2;
154         digits[0] = bpm / 10; // 十の位
155         digits[1] = bpm % 10; // 一の位
156     } else {
157         numDigits = 1;
158         digits[0] = bpm; // 一の位のみ
159     }
160
161     // 横方向中央寄せの計算

```

```

162 // 各桁は幅FONT_WIDTH(3), 桁間スペース1px →合計幅 = numDigits*3 + (numDigits-1)*1
163 int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
164 int startX = (MATRIX_COLS - totalWidth) / 2; // 列に対する左端オフセット12
165
166 // 各桁をフレームバッファに描画 (桁ピッチ = 3px + 1間隔px = 4) px
167 for (int i = 0; i < numDigits; i++) {
168     drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
169 }
170
171 // フレームバッファ×ドット(812=96)を uint32_t[3] 形式へパックする.
172 // ドット番号 n = row*12 + (～) を, 側から順に詰めるcol095MSB
173 // (Arduino_LED_Matrix の loadFrame が要求するビット配置).
174 uint32_t bitmap[3] = {0, 0, 0};
175 for (int row = 0; row < MATRIX_ROWS; row++) {
176     for (int col = 0; col < MATRIX_COLS; col++) {
177         if (frameBuffer[row][col]) {
178             int n = row * MATRIX_COLS + col;
179             bitmap[n / 32] |= (1UL << (31 - (n % 32)));
180         }
181     }
182 }
183
184 // マトリクスへ描画LED
185 matrix.loadFrame(bitmap);
186 }
187
188 // drawDigit
189 // 引数 digit : 描画する数字 (～) 09
190 // x_offset : フレームバッファ上の描画開始列 (～) 011
191 // font3x5[digit] の横×縦 35 ビットマップを, frameBuffer の
192 // (～y_offset, ～x_offset) 領域に書き込む.
193 // 縦方向は (MATRIX_ROWS - FONT_HEIGHT)/2 行のオフセットで上下中央寄せ.
194
195 void drawDigit(int digit, int x_offset) {
196     // 範囲外の数字は無視
197     if (digit < 0 || digit > 9) return;
198
199     // 縦方向中央寄せオフセット: 行に行フォントを配置85 → (8-5)/2 = 行余白1
200     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;
201
202     for (int y = 0; y < FONT_HEIGHT; y++) { // 縦ドット5
203         for (int x = 0; x < FONT_WIDTH; x++) { // 横ドット3
204             int col = x_offset + x;
205             int row = y_offset + y;
206             // バッファ境界チェック (端に近い場合の範囲外アクセス防止)
207             if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {
208                 frameBuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
209             }
210         }
211     }
212 }追加
213
214 // 'E'を受信したら全消灯, 'をリセットcurrentBPM
215 void clearDisplay() {
216     currentBPM = -1;
217     uint32_t blank[3] = {0, 0, 0};
218     matrix.loadFrame(blank);
219 }

```